STUDYFLOW

Architecture and implementation

Data model, core code flows and setup guide

Report 01 / Version 1.0.0+1 / November 4, 2026

StudyFlow is a Flutter application for planning courses and tasks, recording focus sessions and tracking study progress. Hive manages local storage; Firebase provides user accounts and cloud backups. The interface is available in Turkish and English.

How it works
Courses, tasks and focus sessions are stored on the device. Cloud transfer consists of full-plan backup and restore operations started by the user. This keeps offline planning independent of account connectivity.

Section	Page
Scope and technology choices	2
Architecture and source files	3
Data model	4
Local storage and transaction handling	5
State management, navigation and language	6
Firebase accounts and data access	7
Timer, external service and user feedback	8
Setup and execution	9
Verification, delivery and development notes	10

Scope and technology choices

The main workflow is to create a course, add a task to it, complete the task and save a focus session. Overview displays daily study time and pending tasks; Progress shows minute totals for the last seven days.

Component	Reason for use
Flutter / Dart	Build Android and web interfaces from a shared codebase; manage forms, navigation and animations together.
Provider / ChangeNotifier	Update screens when the plan or language changes. Local business rules are grouped in the ViewModel.
Hive	Store a small study plan quickly on the device without a separate database server.
Firebase Auth	Provide email and password accounts.
Cloud Firestore	Store each user’s plan backup as a document containing JSON-like data.
HTTP / JSON	Retrieve Türkiye’s public holidays from an external service.
flutter_localizations	Localize Material components, date pickers and standard buttons for Turkish and English.

First-release boundaries

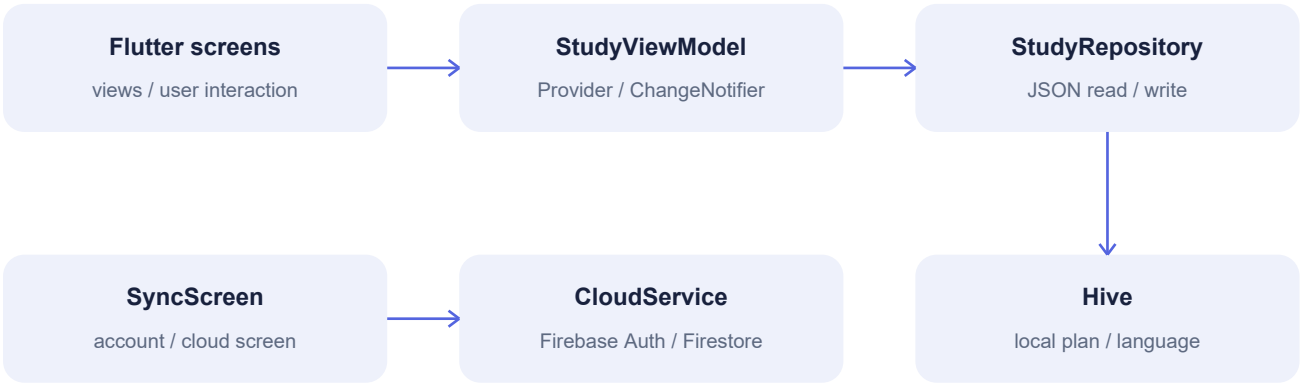
The project is scoped for pair development. Notifications, social rankings, automatic record merging and course-specific focus time are not included. Task completion can be changed; there is no separate editor for task titles, dates or priorities.

Target platforms

The source project includes Android and web targets. The current Android build has been opened in the emulator. An iOS target has not been generated, so iPhone installation and App Store distribution are outside this report’s verification scope.

Architecture and file structure

The local plan follows an MVVM structure. Views collect user input, StudyViewModel performs changes and StudyRepository writes them to Hive. Provider exposes the ViewModel to child screens. The account screen uses CloudService directly, then applies a restored plan to local storage through the ViewModel.



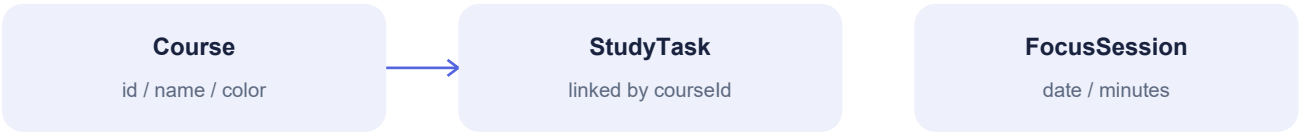
Local plan changes use the ViewModel; account operations use CloudService.

File / folder	Responsibility
lib/main.dart	Hive initialization, Provider tree, theme, supported locales and home screen.
lib/models/study_data.dart	Course, task, session and plan classes; JSON conversion.
lib/viewmodels/study_viewmodel.dart	Add, delete, complete, saving state and daily study-time calculations.
lib/data/study_repository.dart	Read and write the plan and language preference through Hive.
lib/data/cloud_service.dart	Firebase initialization, session, backup, restore and document stream.
lib/data/cloud_errors.dart	Convert Firebase error codes into user-facing messages.
lib/views/	Main tabs, account screen, form dialogs and holiday calendar.
lib/l10n/app_language.dart	Language preference controller, translation dictionary and date formatting.
lib/widgets/operation_overlay.dart	Full-screen loading feedback while an operation is running.

Core tests are in test/. Report capture tools in integration_test/ and test_driver/ generate real Android screens using separate demonstration data. They do not write to the user's normal plan box.

TECHNICAL REPORT

Data model



One course has many tasks. Focus sessions are independent of courses.

Class	Fields	Description
Course	id, name, color	Course identifier, displayed name and interface color.
StudyTask	id, title, courseId, due, done, priority	courseId identifies a course. Priority is 0 for low, 1 for normal and 2 for high.
FocusSession	date, minutes	A completed study session explicitly saved by the user.
StudyData	courses, tasks, sessions	Three lists representing the whole plan, shared by local and cloud transfer.

lib/models/study_data.dart / StudyTask.toJson

```
Map<String, dynamic> toJson() => {
  'id': id,
  'title': title,
  'courseId': courseId,
  'due': due.toIso8601String(),
  'done': done,
  'priority': priority,
};
```

Relationships and dates

Tasks store courseId rather than a course’s text name. Deleting a course also removes tasks with that courseId. Focus sessions are independent of courses, so deleting a course does not change historical study minutes. Task dates are stored as ISO 8601 strings and formatted in the selected interface language.

Identifiers use the current time in microseconds. This is a simple approach for a single-device course project; heavier concurrent creation would benefit from an independent identifier scheme such as UUIDs.

Local storage and update flow

Startup initializes Hive and opens the study_flow String box. The data key contains the JSON plan; language stores the tr or en language code. Language preference is separate from the plan backup, so restoring a backup does not change the interface language.

lib/data/study_repository.dart / persistent storage

```
class StudyRepository {
  StudyRepository(this.box);
  final Box<String> box;
  String loadLanguage() => box.get('language', defaultValue: 'tr')!;
  Future<void> saveLanguage(String code) => box.put('language', code);
  StudyData load() {
    final raw = box.get('data');
    return raw == null
        ? StudyData.empty()
        : StudyData.fromJson(jsonDecode(raw));
  }

  Future<void> save(StudyData data) =>
      box.put('data', jsonEncode(data.toJson()));
}
```

Completing a change

lib/viewmodels/study_viewmodel.dart / _commit

```
Future<bool> _commit(void Function() update) async {
  if (!ready || busy) {
    return false;
  }
  final backup = StudyData.fromJson(data.toJson());
  busy = true;
  error = null;
  update();
  notifyListeners();
  try {
    await repository.save(data);
    return true;
  } catch (_) {
    data = backup;
    error = 'Değişiklik kaydedilemedi. Lütfen tekrar dene.';
    return false;
  } finally {
    busy = false;
    notifyListeners();
  }
}
```

The plan is copied before saving. If a write fails, the previous copy is restored; the busy flag is cleared in finally. Invalid JSON on initial load blocks normal editing and displays an error, preventing a corrupt record from being silently replaced with a new empty plan.

Screen state and language support

Tabs and navigation

The home page uses IndexedStack for Overview, Tasks, Focus and Progress. This preserves the timer's screen state when changing tabs. Account and holiday screens open as new Navigator routes. Course and task forms use DialogRoute; text controllers are disposed after the closing animation finishes.

Turkish / English selection

lib/l10n/app_language.dart / LanguageController

```
class LanguageController extends ChangeNotifier {
  LanguageController(this.repository)
    : _code = repository.loadLanguage() == 'en' ? 'en' : 'tr';
  final StudyRepository repository;
  String _code;
  Locale get locale => Locale(_code);
  Future<void> setLanguage(String code) async {
    if (!['tr', 'en'].contains(code)) {
      return;
    }
    final previous = _code;
    _code = code;
    notifyListeners();
    try {
      await repository.saveLanguage(code);
    } catch (_) {
      _code = previous;
      notifyListeners();
      rethrow;
    }
  }
}
```

Language is selected from the icon at the top right. MaterialApp uses the selected locale. GlobalMaterialLocalizations delegates localize the date picker and standard buttons. Screen text comes from a translation dictionary; course and task names entered by users are not translated.

Filtering and calculations

Tasks can be searched by title and filtered as pending, done or all. Sorting uses due date first and priority when dates match. Progress sums session minutes on the same calendar day. Course completion is completed tasks divided by total tasks; courses without tasks display zero.

Firebase and access rules

Firebase settings are passed at build time through firebase.local.json. The Android application ID is com.testkullanicisi.study_flow. An empty authDomain is passed as null, preventing Android Auth errors caused by an empty custom domain.

lib/data/cloud_service.dart / plan backup

```
DocumentReference<Map<String, dynamic>> get document => FirebaseFirestore
    .instance
    .collection('users')
    .doc(user!.uid)
    .collection('backups')
    .doc('planner');
Future<void> upload(StudyData data) async {
  await document
      .set({'data': data.toJson(), 'updatedAt': FieldValue.serverTimestamp()})
      .timeout(const Duration(seconds: 20));
}
```

The backup is stored in users/{uid}/backups/planner with data and updatedAt fields. Upload replaces the complete cloud plan. Download reads the server document and replaces the local plan. Both operations require confirmation. Automatic two-way synchronization and per-record conflict resolution are not implemented.

firestore.rules / user-owned document access

```
rules_version = '2';
service cloud.firestore {
  match /databases/{database}/documents {
    match /users/{userId}/backups/{backupId} {
      allow read, delete: if request.auth != null && request.auth.uid == userId;
      allow create, update: if request.auth != null && request.auth.uid == userId
        && request.resource.data.keys().hasOnly(['data', 'updatedAt'])
        && request.resource.data.data is map
        && request.resource.data.updatedAt == request.time;
    }
  }
}
```

The rules check ownership and top-level fields. They do not validate every nested list item’s schema. Security verification must separately attempt access to another user’s document.

Timer and external service

Focus sessions

Users can select 15, 25 or 45 minutes. Remaining time is calculated from the difference between the end time and the current clock, accounting for time spent in the background. A completed session is added to the plan with Save session.

lib/views/home_screen.dart / clock-based timer

```
void tick() {
  if (endsAt == null || !mounted) {
    return;
  }
  final seconds =
      (endsAt!
                  .difference(context.read<StudyViewModel>().now())
                  .inMilliseconds /
              1000)
          .ceil();
  setState(() {
    remaining = math.max(0, seconds);
    if (remaining == 0) {
      timer?.cancel();
      endsAt = null;
      completed = true;
    }
  });
}

void start() {
  if (completed) {
    return;
  }
  endsAt = context.read<StudyViewModel>().now().add(
    Duration(seconds: remaining),
  );
  timer = Timer.periodic(Duration(seconds: 1), (_) => tick());
  setState(() {});
}
```

Holiday calendar

The calendar retrieves the year’s public holidays in Türkiye from Nager.Date. It displays localName in Turkish and name in English. Requests have a 12-second timeout and a retry option. The list is not stored for offline use.

GET endpoint

```
https://date.nager.at/api/v3/PublicHolidays/{year}/TR
```

Loading and error feedback

Firebase operations display OperationOverlay and block background input and back navigation. On failure, the overlay closes and a localized message appears. A network timeout does not guarantee that the server operation was cancelled; check its result before retrying.

Setup and execution

The verified development environment is Windows with Flutter 3.47.9 and Dart 3.13.7. Android Studio supplies the SDK and Java components. The first build may download missing SDK platforms, NDK or CMake components. Run Flutter commands from the study_flow directory.

Initial checks

```
cd "<project-folder>/study_flow"
flutter pub get
flutter doctor
flutter devices
flutter emulators
```

Launch the emulator and Firebase-configured app

```
flutter emulators --launch Medium_Phone_API_37.0
flutter run -d emulator-5554 --dart-define-from-file=firebase.local.json
```

Firebase setup sequence

Add an Android app to the Firebase project with package name com.testkullanicisi.study_flow. Place the downloaded google-services.json in android/app/. Enable Email/Password under Authentication. Create a Standard edition (default) Firestore database and publish firestore.rules. The project's firebase.local.json contains the corresponding Android application settings.

Variable	Source
FIREBASE_API_KEY	API key for the selected Firebase application.
FIREBASE_APP_ID	Android mobilesdk_app_id value.
FIREBASE_MESSAGING_SENDER_ID	Firebase project number.
FIREBASE_PROJECT_ID	Firebase project identifier.
FIREBASE_AUTH_DOMAIN	May be empty on Android; use the appropriate web app domain for web.

firebase.local.json is excluded from version control. Reports and screenshots do not contain real keys. Web execution requires separate Firebase web application settings rather than the Android app ID.

Verification and delivery outputs

Purpose	Command / output
Code analysis	flutter analyze
Unit and widget tests	flutter test
Android screenshot capture	flutter drive --driver=test_driver/report_driver.dart --target=integration_test/report_capture_test.dart -d emulator-5554 --dart-define-from-file=firebase.local.json
Android test APK	flutter build apk --debug --dart-define-from-file=firebase.local.json
APK location	build/app/outputs/flutter-apk/app-debug.apk
Web preview	flutter run -d chrome Specify a separate web configuration file where required.

Verification summary

In the November 4, 2026 run, all eight tests under test/ passed and flutter analyze reported no issues. Android screenshot capture ran in a separate demonstration box. These tests do not prove every negative account or backup scenario against the real Firebase service; the third report details that scope.

Distribution and known limitations

The APK uses a debug signature. Store distribution requires release signing, version management and publishing configuration. An active timer is not persisted when the app terminates. Local data has no application-level encryption. Keeping the entire plan in one Firestore document is simple for small datasets; document limits and concurrent editing should be reassessed as data grows.

Sources

This report is based on the source code, pubspec.yaml, firestore.rules and tests. Code excerpts are taken from those files. Technical reference addresses:

https://docs.flutter.dev/

https://firebase.google.com/docs/flutter/setup

https://firebase.google.com/docs/firestore/security/rules-conditions

https://docs.flutter.dev/ui/internationalization

https://date.nager.at/api/v3/PublicHolidays/2026/TR